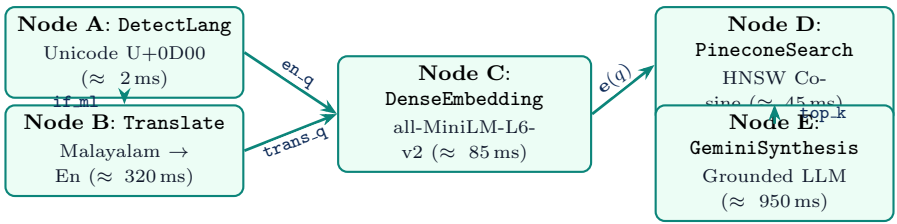


Visual Data-Flow Graph & Execution Timing



Pipeline Phase 1: Input audit → Unicode detection → Translation (if needed)
Pipeline Phase 2: 384-dimensional dense semantic query embedding
Pipeline Phase 3: Pinecone sub-50ms HNSW approximate nearest neighbor search
Pipeline Phase 4: Gemini grounded generation with statutory citations (≈ 1.4 s total)

Computer Science Explanation (Asynchronous Execution Pipeline):

- **Vertices (V) = Asynchronous Transformation Tasks:**
  Node A: `is_ml = detect_unicode(raw_query, 0x0D00, 0x0D7F)` *[Language Router, ≈ 2 ms]*
  Node B: `q_en = await gemini.translate(raw_query, to="en")` *[Translation Worker, ≈ 320 ms]*
  Node C: `emb = sbert_model.encode(q_en, normalize=True)` *[Dense Vector Embedder, ≈ 85 ms]*
  Node D: `chunks = await pinecone.query(emb, top_k=5)` *[Managed Vector Index, ≈ 45 ms]*
  Node E: `answer = await gemini.generate(prompt(q_en, chunks))` *[Grounded LLM Synthesis, ≈ 950 ms]*
- **Edges (E) = Data Dependencies & Variable Passing:**
  Edge $\langle A, B\rangle$ routes Malayalam strings; edge $\langle B, C\rangle$ and direct branch $\langle A, C\rangle$ provide normalized English text.
  Edge $\langle C, D\rangle$ transmits the 384-d float vector $\mathbf{e}(q)$; edge $\langle D, E\rangle$ transfers statutory passages with metadata.
- **Latency Profile & System Responsiveness:**
  Non-blocking asynchronous I/O ensures sub-200ms latency for cached vector queries and ≈ 1.4 s total round-trip for full LLM generation.